Raspberry-Pi-Pico-2-W User Guide

User guide | Ask a Question

Price: $8.99-$12.8
Model: Raspberry Pi Pico 2 W (RP2350) Pa...
Brand: Raspberry Pi
SKU: SDP20002

[Tutorial Navigation]
Overview
Introduction
Features
Pinout Definition
Dimensions
Pico Getting Started
Firmware Download
Basic Introduction
MicroPython Series
Install Thonny IDE
Pinout Function
MicroPython Series Tutorials
C/C++ Series
Install VSCode
Install Extension
Configure Extension
New Project
Compile Project
Flash Firmware
Import Project
Update Extension
Install Arduino IDE
Install Arduino IDE
Arduino IDE Interface
Install Arduino-Pico Core in Arduino IDE
Upload Demo at the First Time
Open Source Demo
Expansion Experiment
External LED Experiment
Traffic Light System Experiment
Burglar Alarm LED Buzzer Experiment
Potentiometer Experiment
WS2812 Experiment
Ultrasonic IOC Experiment
Resources
Official Resources
Demo
Pico 2 W
Schematic diagram and datasheets
Software library
Pinout
User manuals
Schematic diagram and datasheets
Related books
Raspberry Pi open source demos
FAQs
Question: What are the common issues and precautions when using Raspberry Pi Pico 2 (RP2350) Raspberry Pi Pico 2 W for development?
Question: Is there interfaces with Raspberry Pi Pico 2 W?
Question: Raspberry Pi Pico 2 (RP2350) Raspberry Pi Pico 2 W pins are all who does leading IO show a high voltage level when the pin is left suspended?
Support

Overview

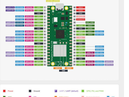
Introduction

Raspberry Pi Pico 2 W is an official Raspberry Pi-designed, low-cost yet flexible development platform based on the RP2350, which is a wireless upgrade of the Raspberry Pi Pico. It features all the characteristics of the Pico2 with the addition of a 2.4GHz wireless interface.

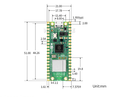
Features

- RP2350 microcontroller chip designed by Raspberry Pi
- Dual-core Arm Cortex-M33 and Hazard3 processors, clocked at 150 MHz
- 520 KB of on-chip SRAM and 4MB of on-chip flash
- Supports Wi-Fi 4 wireless network and Bluetooth 5.2
- Supports USB 1.1 host and device, supporting directly to further boards
- USB1.1 host and device support
- Supports low-power sleep and dormant modes
- Drag-and-drop programming using mass storage USB recognition on mass storage
- 26 multi-function GPIO pins
- 2 SPI, 2 I2C, 2 UART, 3 ADC, 16 PWM channels
- 1 accurate on-board temperature sensor
- 4 programmable input/output state machines for custom peripheral support

Pinout Definition



Dimensions



Pico Getting Started

Firmware Download

- MicroPython Firmware Download

- C,Blink Firmware Download

Basic Introduction

Raspberry Pi Pico Backup

MicroPython Series

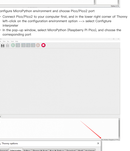
Install Thonny IDE

In order to facilitate the development of Pico/Pico2 boards using MicroPython on a computer, it is recommended to download the Thonny IDE

- Download Thonny IDE and follow the steps to install, the installation packages are all next-next-finish, Thonny IDE supports all Pico/Pico2 series of boards for other versions.
  - Thonny IDE official download link
  - Thonny IDE download link-1
  - Thonny IDE download link-2

After installation, the language and motherboard environment need to be configured for the first use. Since we are using Pico/Pico2, pay attention to selecting the Raspberry Pi option for the motherboard environment.

- Configure MicroPython environment and choose Pico/Pico2 port
  - Connect Pico/Pico2 to your computer first and in the lower right corner of Thonny left click on the configuration environment option --> select Configure interpreter...
  - In the pop-up window, select MicroPython (Raspberry Pi Pico), and choose the corresponding port



Flash Firmware

- Click OK to return to the Thonny main interface, download the corresponding firmware library and burn it to the device, and then click the Stop button to display the Shell interface of the Pico/Pico2.
- **Note: Flashing the Pico2 firmware provided by Micropython may cause the device to be unresponsive, please use the Firmware below or in the package provided by Raspberry Pi.**
  - Pico2 Firmware library-1
- How to download the Firmware library for Pico/Pico2 in windows: After holding down the BOOTSEL button on the Pico/Pico2 board and connecting it to the USB port, a removable disk will appear on the computer, copy the Firmware library into it.
- How to download the Firmware library for RP2040/RP2350 in windows: After holding down the BOOTSEL button on the Pico/Pico2 board and connecting it to the USB port, release the BOOTSEL button and then release the RESET key, a removable disk will appear on the computer, copy the Firmware library into it (you can also use the Pico/Pico2 method)

MicroPython Series Tutorials

- [MicroPython] machine.Pin class function detailed
- [MicroPython] machine.Timer class function detailed
- [MicroPython] machine.ADC class function detailed
- [MicroPython] machine.UART class function detailed
- [MicroPython] machine.I2C class function detailed
- [MicroPython] machine.SPI class function detailed
- [MicroPython] rp2 module function class function detailed

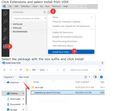
C/C++ Series

For C/C++, it is recommended to use Pico VSCode for development. This is a Microsoft Visual Studio Code extension designed to make it easier for you to create, develop, and debug projects for the Raspberry Pi Pico series development boards. No matter if you are a beginner or an experienced professional, this tool can assist you in developing Pico with confidence and ease. Here's how to install and use the extension:

- official website tutorial: https://www.raspberrypi.com/news/pico-vscode-extension/
- This extension is suitable for Raspberry Pi Pico, Pico2 and the RP2040 and RP2350 series development boards developed by Raspberry Pi.
- The development environment defaults to Windows7. For other environments, please refer to the official tutorial for installation.

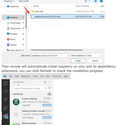
Install VSCode

1. First, click to download pico-vscode.zipapp.zip and open the package, double-click to install VSCode



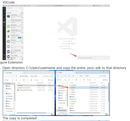
Install Extension

1. Click Extensions and select Install from VSIX

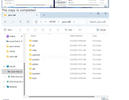


2. Select the package with the vsix suffix and click Install

3. Then vscode will automatically install raspberry-pi-pico and its dependency extensions, you can click Refresh to check the installation progress



4. Finally, if the right lower corner shows that the installation is complete, Close



Configure Extension

1. Open directory C:\Users\username and copy the entire .pico-sdk to that directory

2. The copy is completed

3. Open vscode and configure the paths for the Raspberry Pi Pico extension

The configuration is as follows:

```
Toolchain Path:
D:\Users\.pico-sdk\tools\1.13.0_6/bin\cmake.exe

Ninja Path:
D:\Users\.pico-sdk\ninja\v1.12.1/ninja.exe

Python3 Path:
D:\Users\.pico-sdk\python\3.12.1/python.exe
```

New Project

- After configuration is complete, create a new project, enter the project name, select the path, and click Create to create the project.
  - To test the official example, you can click on the Example next to the project name.

- The project is created successfully

Compile Project

1. Select the SDK version

2. Select Yes for advanced configuration

3. Choose the cross-compilation chain. TC13MP is applicable for ARM cores, RISCV 13.3 is applicable for RISCV cores. You can select either based on your requirements.

4. Select Default for CMake version (the path configured earlier)

5. Select Default for Ninjaversion

6. Select the development board

7. CD projects to compile

8. **Firmware is successfully compiled**

Flash Firmware

Here are two methods for flashing firmware:

1. Flash firmware using the pico-vscode plugin
   Ensure the development board is in the computer, click Run to flash the firmware directly

2. Flash the firmware manually

   1. Press and hold the Boot button
   2. Connect the development board to the computer
   3. The computer system will recognize the development board as a USB device.
   4. Copy the .uf2 file to the USB drive, and the device will automatically restart, indicating successful program flashing.

Import Project

1. Select the project directory and import the project

2. If errors like of the imported project cannot have Chinese (including comments), otherwise the import may fail
3. To import your own project, you need to add a line of code to the CMake file to specify the board type, otherwise the compilation error. If project is imported, the corresponding firmware will still be suitable for pico

```
set(PICO_BOARD pico CACHE STRING "Board type")
```

Update Extensions

1. Ensure extension version in the offline package is 0.9.0, and you can also choose to update to the latest version after the installation is complete

Install OS Debian

Install Arduino IDE

1. First, go to Arduino official website to download the installation package of the corresponding version

Downloads

Support the Arduino IDE

2. Open the downloaded installation package and install

Notice: During the installation process, it will prompt you to install the Arduino USB driver, click Install

Arduino IDE Local Language

1. After the first installation, when you open the Arduino IDE, it will be in English. You can switch to other languages in File --> Preferences, or continue using the English version

2. In the Language field, select the language you want to switch to, and click OK.

Install Arduino-Pico Core in Arduino IDE

1. Open the Arduino IDE, click on the File in the top left corner, and select Preferences

2. Add the following link to the attached board manager URL, and then click OK

```
https://raw.githubusercontent.com/earlephilhower/arduino-pico/release/package_rp2040_index.json
```

Note: If you already have an ESP32 board URL, you can use a comma to separate the URLs as follows:

```
https://dl.espressif.com/dl/package_esp32_index.json,https://raw.githubusercontent.com/earlephilhower/arduino-pico/release/package_rp2040_index.json
```

3. Click Tools > Development Board > Board Manager > Search pico, ensure the computer has already been installed. It shows that it is installed

Upload Demo at the First Time

1. Connect the device to the USB port. Hold down the Pico board's connect the pico to the USB port of the computer via the Micro USB cable, and release the button after the computer recognizes a removable hard disk (RPI-RP2).

2. Download the program and open 03-LED.ino under the arduino\PWM\03-LED

3. Click Tools --> Port; remember the existing COM; do not click this COM (The COM displayed is different on different computers, remember the COM on your own computer)

4. Connect the driver board to the computer using a USB cable. Then, go to Tools > Port, and select the corresponding COM port. After adjusting, when you connect again, an additional COM port will appear.

5. Click Tools > Development Board > Raspberry Pi Pico > Compatible versions Raspberry Pi Pico, Raspberry Pi Pico 2, etc.

6. After setting it up, click the right arrow to upload the program

- **If issues arise during this period, and if you need to reinstall or update the Arduino IDE version, it is necessary to uninstall the Arduino IDE completely. After uninstalling the software, you need to manually delete all remaining entries in the installation directory to ensure a clean installation. Failure to do so may lead to issues when the new version is installed. Then, proceed with a fresh installation.**

Reference

MicroPython video demo (github)

MicroPython Firmware/Write demos (CD)

Raspberry Pi official C/C++ demo (github)

Raspberry Pi official micropython demo (github)

Arduino official C/C++ demo (github)

Example Experiments

Download the Sensor Kit on your computer desktop to conduct some interesting experiments.

External LED Experiment

- Connect the hardware according to the Fritzing Diagram, connect the Micro USB cable to the computer, open the Thonny IDE, and write the code for the external LED in Thonny, and you can see that the red light is flashing when you run the demo.
- Precautions for use: The longer pin of the LED is the positive pole, the shorter one is the negative pole. The resistance connected to the LED output port, the resistance should be connected to the GPIO output port, and the resistor must be connected when using.

External LED Wiring

- Code analysis

```
led_external = machine.Pin(15, machine.Pin.OUT) #Set GPIO to output mode
while True:
    led_external.toggle() #Change the state of the LED light
    time.sleep(0.5)
    utime.sleep(1)
```

Traffic Light System Experiment

- Connect the hardware according to the Fritzing Diagram, connect the Micro USB connected to the computer, open the python file in the demo Lesson 5-Traffic Light System in Thonny, and the demo will run, the traffic light is working normally, and the buzzer will be triggered when the button is pressed.
- Precautions for use: The longer pin of the LED is the positive pole, the shorter pin is the negative pole. The resistance connected to the LED output port, the resistance should be connected to the GPIO output port, and the resistor must be connected when using. The red wire of the buzzer is connected to the GPIO port output, and the black wire is connected to GND.

- Code analysis

```
def button_reader_thread():    #detect if a button is pressed
    global button_pressed
    while True:
        if button.value() == 0:
            button_pressed = True

_thread.start_new_thread(button_reader_thread, ()) #Run the button detect keystroke
while True:
    if button_pressed == True: #If the button is pressed, the red light and the buzzer alarms
        led_red.value(1)
        for i in range(50):
            buzzer.value(1)
            utime.sleep(0.2)
            buzzer.value(0)
            utime.sleep(0.2)
        button_pressed = False
    else:    #Traffic light cycle
        print("Traffic light cycle")
        led_green.value(1)
        for i in range(10):    #When the light turns from green to red, the green light goes off first, the yellow light is on for two seconds, then the yellow light and red light will go off, and the red light will be on.
            utime.sleep(0.5)    #When the light turns from green to red, the green light goes off first, the yellow light is on for two seconds, then the yellow light and red light will go off, and the red light will be on.
            utime.sleep(0.5)
        led_yellow.value(1)
        led_green.value(0)
        utime.sleep(1)
        utime.sleep(1)
        led_green.value(0)
        led_red.value(1)
        utime.sleep(1)
        utime.sleep(1)
        led_red.value(0)
```

Bumper Alarm LED Buzzer Experiment

- Connect the hardware according to the Fritzing Diagram, connect the Micro USB connected to the computer, open the python file in the demo Lesson 6-Bumper Alarm LED Buzzer in Thonny, run the demo and you can see that when you move in front of the Passive Infrared sensor, the LED light will flash and the buzzer will sound an alarm.
- Precautions for use: The middle pin of the Passive Infrared sensor is the data output pin, and the other two sides can be connected to VCC and GND respectively.

- Code analysis

```
def pir_handler(pin):    #Interrupt handling function, the buzzer sounds, the LED rapidly flashes
    print("ALARM! Motion detected!")
    led_red.value(1)
    led_toggle()
    buzzer.toggle()
    time.sleep(0.1)
sensor_pir.irq(trigger=machine.Pin.IRQ_RISING, handler=pir_handler) #Turn on the interrupt, and when the human body is sensed, the program will enter the interrupt handling function for processing.
while True:  #If there is no exception, the state of LED will be flashing and the buzzer
    led_toggle()
    utime.sleep(1)
```

Potentiometer Experiment

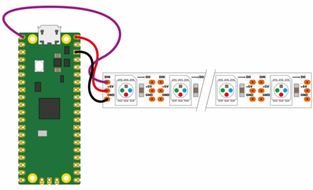
- Connect the hardware according to the Fritzing Diagram, connect the Micro USB connected to the computer, open the python file in the demo Lesson 10-Potentiometer in Thonny, run the demo, and rotate the potentiometer to see that the brightness of the red light changes, and the buzzer sounds.
- Precautions for use: The middle pin of the Potentiometer is the data output pin, and the pins on both sides can be connected to VCC and GND respectively.

- Code analysis

```
potentiometer = machine.ADC(26) #Use GPIO26 as the analog signal collection pin
conversion_factor = 3.3 / 65535
while True:
    voltage = potentiometer.read_u16() * conversion_factor
    print("Potentiometer voltage: {:.2f}V".format(voltage))
    print(voltage) #Print the voltage information, the voltage value will change as the sliding variator rotates
    utime.sleep(1)
```

WS2812 Experiment

- Connect the hardware according to the figure below, connect the Micro USB connected to the computer, open the WS2812_RGB_LED.py file in the demo Lesson-25 WS2812 in Thonny, run the demo to see the RGB colors of blue, red, green, and white at once.



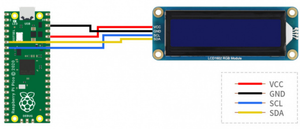
- Code analysis

```
#This code uses the state machine mechanism, the following code
is a decorator, in which we can initialize the hardware, set the
level of the pins, etc.
#label("bitloop") We can define labels in the code to facilitate
our execution by jumping to them.
#jmp(not_x,"do_zero") When x=0, we adjust to the label
"do_zero".
#nop() .set(0) [T2 - 1] When x=0, it will jump here to execute.
@asm_pio(sideset_init=PIO.OUT_LOW, out_shiftdir=PIO.SHIFT_LEFT,
autopull=True, pull_thresh=24)
def ws2812():
    T1 = 2
    T2 = 5
    T3 = 1
    label("bitloop")
    out(x, 1)               .side(0)    [T3 - 1]
    jmp(not_x, "do_zero")   .side(1)    [T1 - 1]
    jmp("bitloop")          .side(1)    [T2 - 1]
    label("do_zero")
    nop()                   .side(0)    [T2 - 1]
```

```
# Create the StateMachine with the ws2812 program, outputting on
Pin(22).
sm = StateMachine(0, ws2812, freq=8000000, sideset_base=Pin(0))
#Create a state machine
# Start the StateMachine, it will wait for data on its FIFO.
sm.active(1) #Start state machine
# Display a pattern on the LEDs via an array of LED RGB values.
ar = array.array("I", [0 for _ in range(NUM_LEDS)])
print(ar)
print("blue")
for j in range(0, 255):
    for i in range(NUM_LEDS):
        ar[i] = j
    sm.put(ar,8)  The method of #put() is to put the data into
the output FIFO of the state machine
    time.sleep_ms(5)
```

LCD1602 I2C Experiment

- Connect the hardware according to the figure below, connect the Micro USB connected to the computer, open the python file in the demo Lesson-21 LCD1602 I2C in Thonny, save the RGB1602.py file as a Raspberry Pi Pico, and run the Choose_Color.py to see a different color switch every 5 seconds; run Discoloration.py file to see the RGB color gradient effect.



- Code analysis

Choose_Color.py

```
#Define color
rgb9 = (0,255,0) #Green
lcd.setCursor(0, 0) #Set cursor position
# print the number of seconds since reset:
lcd.printout("Waveshare") #Write characters
lcd.setCursor(0, 1) #Set the cursor position to the second row
and the zero column
lcd.printout("Hello,World!") #Write characters
lcd.setRGB(rgb1[0],rgb1[1],rgb1[2]); #Set the backlight
```

Discoloration.py

```
t=0
while True:

    r = int((abs(math.sin(3.14*t/180)))*255); #RGB changes over
time
    g = int((abs(math.sin(3.14*(t+60)/180)))*255);
    b = int((abs(math.sin(3.14*(t+120)/180)))*255);
    t = t + 3;
    lcd.setRGB(r,g,b); #Reset the RGB value
# set the cursor to column 0, line 1
    lcd.setCursor(0, 0) #Navigate to the first row and the zero
column
# print the number of seconds since reset:
    lcd.printout("Waveshare") #Write characters
    lcd.setCursor(0, 1) #Navigate to the second row and the zero
column
    lcd.printout("Hello,World!") #Write characters
    time.sleep(0.3)
```

Resources

Official Resources

Demo

- Raspberry-Pi-Pico-Basic-Kit Demo
- Raspberry-Pi-Pico-Sensors_Pack Demo
- Raspberry-Pi-Pico-Sensors_Kit Demo

Pico 2 W

Schematic diagram and datasheets

- Pico 2 W Schematic diagram
- Pico 2 W Pinout definition
- Pico 2 W Datasheet
- Pico 2 W Networking manual

Firmware library

- Pico 2 W Firmware (Micropython)

Pico2

User manuals

- Pico2 Getting Started
- Pico2 C SDK User Manual
- Pico2 Python SDK User Manual

Schematic diagram and datasheets

- Pico2 Schematic diagram
- Pico2 Pinout definition
- Pico2 Datasheet
- RP2350 Datasheet
- RP2350 Hardware Design Reference Manual

Related books

- Get Started with MicroPython on Raspberry Pi Pico
- Raspberry Pi related books download

Raspberry Pi open source demos

- Raspberry Pi official C/C++ Demos (github)
- Raspberry Pi official micropython Demos (github)

FAQ

Question: What are the common issues and precautions when using Raspberry Pi Pico 2 and Raspberry Pi Pico-2-W for development?

Answer:

You can refer to the official manual Pico FAQs

Question: Is there interference with Raspberry Pi Pico 2 W?

Answer:

1. The pin that sets the input state must be initialized to pull up or down
2. Replace the USB cable and see if the USB cable is the one causing the problem

Question: Raspberry Pi Pico 2 GPIO is configured as a pull-down input, why does reading IO show a high voltage level when the pin is left unconnected?

Answer:

You can refer to the RP2350-E9 section in the RP2350 Datasheet

Support

Monday-Friday (9:30-6:30) Saturday (9:30-5:30)
Email: services01@spotpear.com

TAG: Milk-V Duo S Usage of USB Type A interface Robotic Arm Control Intel 11th ESP32-S3 1.28inch Jetson Orin Nano/NX Super Developer IO Base Development Board Dual network RJ45 For Jetson Orin Nano/NX Module ESP32-C6 WiFi 6 Development Board ESP32-C6-MINI-1Support WiFi6 Bluetooth5 Zigbee Thread XIAO RA4M1 Tiny SuperMini Board For Arduino Uno IDE For SeeedStudio R7FA4M1 Raspberry Pi Pico 240x280 PC USB Monitor Screen VGA to TTL ESP32 ST7789 ESP32 3.5 inch Capacitive TouchScreen LCD Display ST7796 FT6336 320x480 For Arduino/Raspberry Pi/Pico Raspberry Pi PCM5122 I2S DAC HiFi Sound Card DAC Audio expansion Board Free-Driver For Pi5/Pi4/Zero DeepSeek AI Voice Chat ESP32-S3 Box N16R8 AUDIO Board Support External LCD/Camera AI deepseek VGA to LVDS Tuya AI Voice Robot T5-E1 1.75 inch Round AMOLED TouchScreen GPS For XiaoZhi Development Board UNO Minima RA4M1 ZERO Tiny SuperMini R7FA4M1 Board Compatible With R4 For Arduino Raspberry Pi 15.6 inch QLED Quantum Dot Display 1920x1080 Capacitive Touchscreen For Computer PC/Jetson/Game $167.99 ADXL354CEZ

©Copyright 2015 Berlinger Haus | All Rights Reserved

Address: Rm401, building 4, Enlai industrial city, Nankang Road, Danzhutou community, Nanwan street, Longgang District, Shenzhen
Email: tech-support@spotpear.com (Only tech support .Order see 'CONTACT US' at our web)